INF1103: Lab 9

Design Patterns and Error Handling
(Strategy, Factory, Validation, Try / Except and Automated Tests)

Introduction

This lab follows last week's Lab 8. Will follow a similar format of ***Understanding*** (Section 1) before you write any code and then implementation (Section 2) to put your understanding into practice and develop those OOP skills further.

We will also continue to use ***'Predict First'*** questions, to think about your code flow and outputs before you run your code. This approach develops your thinking, understanding and awareness of code flow which not only helps develop your OOP skills but also can help you with similar questions in your quiz. For each predict first, fill out your answers in a Word document and submit it with all other files for this lab.

In Lab 8, you modelled suppliers, inventories and customer orders, deciding whether each relationship was an association, aggregation or composition. You also used type hints, applied the Liskov Substitution Principle and used polymorphism through a common Product type.

In this lab, you will improve the design and robustness of your Inventory Management System. You will use the Strategy pattern to make order discounts interchangeable, and the Factory pattern to centralise how products are created. You will then add validation and exception handling, so your program fails gracefully when something unexpected happens, such as invalid input or a missing file.

Finally, you will write automated tests for your new code. These are the kind of tests a CI pipeline runs every time you push to GitHub.

Hint from the lecture: use the simplest design that solves the current problem best. A pattern is a design decision, not a rule.

Learning outcomes

Explain what a software design pattern is and when one is worth using.
Apply the Strategy pattern to make behaviour interchangeable without changing the class that uses it.
Apply the Factory pattern to centralise object creation.
Distinguish validation from exception handling and use each appropriately.
Handle expected exceptions specifically, so the program fails gracefully.
Write automated tests that could run in a CI pipeline.

1. Review design patterns and error handling (Thinking / Understanding / No Coding)

Before changing your system, review the ideas from the lecture. Keep one question in mind throughout the lab: what problem am I solving, and is this the simplest design that solves it?

1.1 Design patterns - proven solutions to recurring problems

A design pattern is a general, reusable solution to a commonly occurring design problem. It describes how classes and objects are structured and collaborate. It is a solution template, not finished code, and not a library you import.

Patterns give developers a shared vocabulary for discussing design choices.
Patterns often separate responsibilities, which improves maintainability.

Patterns should not be applied blindly. A pattern can add complexity to a problem that a simple solution would solve.

1.2 Strategy - interchangeable behaviour

The Strategy pattern solves the problem of one task that can be performed in several different ways. The obvious solution is a large if/elif block, but every new option means editing that method, which breaks the Open/Closed Principle (OCP).

Each option (behaviour) moves into its own class.
All strategies provide the same method, usually defined by an abstract class.
The class that uses a strategy only calls that shared method. It does not know how each strategy works.
A new option means a new class, not a change to the class that uses it.

1.3 Factory - centralised object creation

The Factory pattern solves the problem of object creation being repeated or complicated across a program. Without it, client code must know about every concrete class, and must change whenever a new type is added.

The factory owns the creation decision. The rest of the program asks the factory for an object.
If creation rules change, there is one place to update.
The factory itself still changes when a new type is added. As discussed in the lecture, that is acceptable for this module, as long as you can explain the decision (basically to avoid over complicating code for a small application!)

1.4 Validation and exception handling - working together

Validation and exception handling are not replacements for each other.

Validation checks that data meets expected rules before it is processed, usually with if statements. For example, a quantity must be greater than zero.
Exception handling responds to an operation that raises an exception, using try / except. For example, converting text to a number, or opening a file that may not exist.
Catch exceptions at a level where you can make a useful decision, keep messages simple for users, and be specific about which exception you catch.
Do not continue with incomplete data after an exception. Ask the user to try again until you have what you need.

1.5 Look at your own code

Diagnose: Before writing anything new, look through your Lab 8 code and note down: (1) any if/elif block that chooses between different behaviours, (2) every place you create a PhysicalProduct or DigitalProduct directly, and (3) one place where bad input or a missing file could crash your program. Keep these notes; you will come back to them in Section 6.

2. Implement the Strategy pattern: order discounts

2.1 The problem

The business wants to apply discounts to orders. A first attempt might add a discount type to calculate_total() in your Order class, but don't add this code, it isn't the best approach!

```
def calculate_total(self, discount_type: str = "none") -> float:
    total = sum(line.calculate_subtotal() for line in self.order_lines)
    if discount_type == "none":
        return total
    elif discount_type == "percentage":
        return total * 0.9
    elif discount_type == "bulk":
        return total - 20 if total >= 200 else total
    else:
        raise ValueError("Unknown discount type")
```

Predict first: The business now wants a 5% student discount. Without the Strategy pattern, which class and method must change? After you complete Section 2, answer the same question again for your Strategy version.

Remember, do not add the code above to your project, it is an example of a 'simple way' to solve the problem but not the best way (OCP issues)

2.2 Create the discount strategies

Create an abstract DiscountStrategy class with one abstract method, apply(), that takes an order total and returns the discounted total. Then create three strategies:

`NoDiscount` – returns the total unchanged.
`PercentageDiscount` – takes a percentage when created, e.g. PercentageDiscount(10) takes 10% off.
`BulkDiscount` – takes a threshold and an amount, e.g. BulkDiscount(200, 20) takes 20 off any total of 200 or more.
We have provided the DiscountStrategy base class, which should be inherited by NoDiscount, PercentageDiscount and BulkDiscount

```
class DiscountStrategy(ABC):
    @abstractmethod
    def apply(self, total: float) -> float:
        pass
```

2.3 Use a strategy in Order

Update Order so that it is given a discount strategy, rather than deciding the discount itself. Order should not contain any if/elif logic about discount types.

Order accepts a DiscountStrategy when it is created. If none is given, it uses NoDiscount().
Add a set_discount() method so the strategy can be changed later.

- calculate_total() adds up the order line subtotals and passes the result to the strategy's apply() method.

```
def calculate_total(self) -> float:
    total = sum(line.calculate_subtotal() for line in self.order_lines)
    return self.discount.apply(total)
```

Predict first: Create an order whose lines add up to 250. What will calculate_total() return with BulkDiscount(200, 20)? What if you then call set_discount(PercentageDiscount(10)) on the same order? Write both answers down, then run your code and check.

2.4 Extend without modifying

Add a fourth strategy of your choice, for example a StudentDiscount. You should not need to change Order at all.

Predict first: Look back at Lab 8. Which relationship best describes Order and DiscountStrategy: association, aggregation or composition? Hint: is a discount part of an order, or something the order uses? Could two orders share the same PercentageDiscount object? Write your answer and a multiplicity for each end, then check it against your code.

3. Implement the Factory pattern: creating products

3.1 The problem

At the moment, any part of your program that needs a product creates a PhysicalProduct or DigitalProduct directly. That code must know about every product class, and must change if a new product type is added.

3.2 Create a ProductFactory

Create a ProductFactory class with a static create() method. It takes a product type as a string, plus the values needed to build the product, and returns a Product.

```
class ProductFactory:
    @staticmethod
    def create(product_type: str, product_id: str, name: str, price: float,
               stock_quantity: int, supplier: Supplier) -> Product:
        if product_type == "physical":
            return PhysicalProduct(product_id, name, price, stock_quantity, supplier)
        elif product_type == "digital":
            return DigitalProduct(product_id, name, price, stock_quantity, supplier)
        else:
            raise ValueError(f"Unknown product type: {product_type}")
```

Adjust the arguments to match your own PhysicalProduct and DigitalProduct constructors from Lab 7 and Lab 8.

Then replace the direct product creation you noted in Section 1.5 with calls to ProductFactory.create().

Predict first: What happens if you call ProductFactory.create("service", ...)? Which line raises the exception, and what happens to your program if nothing catches it? Run it and check.

3.3 Who knows about what?

Notice that the factory returns the type Product, not PhysicalProduct or DigitalProduct. The rest of your program now only needs to know about Product and the factory, which reduces coupling and calling methods from the base type variable regardless of which child is in there is polymorphism.

Predict first: Suppose a new BundleProduct type is added. How many places in your code would need to change before you added the factory, and how many after? Count them, then explain why the factory still breaks OCP and why that is acceptable here.

4. Add validation

Validation checks data before it is processed. Add validation with if statements, and raise a ValueError with a clear message when a rule is broken.

A product price must be greater than zero.
A product's stock quantity cannot be negative.
An OrderLine quantity must be greater than zero.

```
if quantity <= 0:
    raise ValueError("Quantity must be greater than zero.")
```

Place each check in the class that owns the rule. For example, the OrderLine should protect its own quantity rather than relying on every caller to check it.

Predict first: Think back to the Lab 8 LSP discussion. If Product validates that stock cannot go negative, should DigitalProduct be allowed to skip or change that rule? Why?

5. Handle exceptions gracefully

5.1 User input

Write a helper function that asks the user for a positive price and keeps asking until it gets a valid value. Converting text with float() can raise a ValueError, and so can your own validation, so handle both.

```
def get_positive_float(prompt: str) -> float:
    while True:
        try:
            value = float(input(prompt))
            if value <= 0:
                print("Please enter a number greater than zero.")
                continue
            return value
        except ValueError:
            print("Please enter a valid number, e.g. 12.50")
```

Predict first: A user enters abc, then -5, then 12.50. Write down exactly what prints after each entry and what value is returned. Then run it and check.

5.2 Loading products from a file

Write a load_products() function that reads products from a CSV file, creates each one through your ProductFactory, and adds it to an Inventory. Use try / except / else / finally:

If the file does not exist, catch FileNotFoundError and print a simple message, without crashing.
If the file opens, the else block reads the rows. Create each product inside its own try / except ValueError, so one invalid row (an unknown product type or a price that is not a number) is reported and skipped, and loading carries on.
After the rows are read, print how many products loaded. Use finally to print "Load attempt finished."

```
def load_products(path: str, inventory: Inventory, supplier: Supplier) -> None:
    try:
        file = open(path, newline="")
    except FileNotFoundError:
        print(f"Products file not found: {path}")
    else:
        with file:
            ...  # read rows with csv.reader and skip the header row
            ...  # for each row: try to create and add the product,
            ...  # except ValueError: report the row and continue
        print(f"{loaded} products loaded.")
    finally:
        print("Load attempt finished.")
```

Use this file, saved as products.csv:

```
product_type,product_id,name,price,stock_quantity
physical,P001,Desk Lamp,25.00,10
digital,D001,Photo Editor,49.90,100
service,S001,Installation,30.00,5
physical,P002,Office Chair,abc,3
physical,P003,Monitor Stand,39.50,7
```

Predict first: Before running, write down how many products will load, which messages will print, and in what order. Then rename products.csv and predict again: which of except, else and finally will run?

6. Update your UML class diagram

Update your UML class diagram from Lab 8 to include:

DiscountStrategy as an abstract class, with your strategies inheriting from it.
The relationship between Order and DiscountStrategy, with multiplicities that match your code.
ProductFactory. It creates products but does not store them, so show it with a dashed arrow to Product labelled «creates» (a dependency).
Your diagram should still make sense to a developer who does not know Python.

7. Testing and review

Test your implementation to confirm that:

Order calculates totals through a DiscountStrategy and contains no discount if/elif logic.
A new discount strategy can be added without changing Order.
The strategy on an existing order can be changed with set_discount().
Products are created through ProductFactory rather than directly.
An unknown product type raises a ValueError with a clear message.
Invalid prices, stock quantities and order quantities are rejected.

Invalid user input is handled without crashing, and the user is asked again.
A missing products file is handled gracefully.
Bad rows in the products file are reported and skipped, and valid rows still load.
Each except catches a specific exception.

8. Reflection task (Optional)

Consider the following questions:

Where in your system would adding a pattern make the design more complicated than it needs to be?
The Factory moved the OCP issue from the client into the factory. Is that a good trade-off for your system? Why?
In 5.2, you skipped bad rows and kept loading. When would rejecting the whole file be the better design decision?
What is the difference between a validation failure and an exception in your code? Give one example of each.
Which of your "Predict first" answers in this lab did not match what happened? What assumption was wrong, and what did you change?
Where in your group project could the Strategy or Factory pattern help, or would it be over-engineering? Why?

9. Git and Docker workflow

As in the previous lab, use your existing Git and Docker workflow to manage and test your project.

Use git status to inspect your working tree.
Stage your project changes using git add .
Commit your changes with a meaningful message.
Push your changes to the remote repository.
Run your program and tests using the established Docker workflow.
Review your commit history using git log.

Submission requirements

Submit your updated Inventory Management System, including:

Your "Predict first" notes: for each, your prediction, what happened, and what you changed if they differed (a few lines each). This can be a simple Word document submitted with the other files.
Python source files, including your tests folder.
Updated UML class diagram showing the Strategy and Factory classes and their relationships.
Screenshot of your Git and Docker workflow as per previous labs.